

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

CASE STUDY

Code of Conduct:

I Atchaya Chandran R(192521394) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

R. Atchay

SIMATS ENGINEERING

Course: Operating Systems (CSA-0409) | Assessment Tool 2

Course Outcome (CO3): Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements.

ASSESSMENT SOLUTIONS MANUAL (10 SELECTED QUESTIONS)

Question 1: Logical vs. Physical Address & Translation Mechanism

A. Concepts and Definitions:

- **Logical Address (Virtual Address):** Generated directly by the CPU during instruction execution. It represents the memory location from the program's perspective and forms the *logical address space*.
- **Physical Address:** The actual hardware address in the main memory (RAM) unit. It represents the physical slot where data resides and forms the *physical address space*.

B. Address Translation Process:

Dynamic address translation at execution time is handled by the hardware **Memory Management Unit (MMU)**:

1. The CPU issues a logical address during an instruction fetch or data access.
2. The MMU intercepts the logical address.
3. The MMU adds the value stored in the **Relocation Register** (Base Register) to calculate the final target address in RAM.

$$\text{Physical Address} = \text{Logical Address} + \text{Relocation Register Value}$$

C. Concrete Example:

If a running process is loaded into RAM starting at physical base address 14000 and the CPU generates a logical address of 345 , the MMU translates this as:

$$\text{Physical Address} = 345 + 14000 = 14345$$

The hardware bus then retrieves data directly from physical RAM location 14345 .

Question 2: Complete Address Translation using Base and Limit Registers

A. Role of Registers:

- **Base Register:** Stores the starting physical RAM address allocated to the process.
- **Limit Register:** Specifies the total size (range) of the logical address space for that process.

B. Translation & Validation Workflow:

1. The CPU generates a logical address offset d .
2. **Boundary Check:** The MMU checks if $d < \text{Limit Register}$.
 - If **False** ($d \geq \text{Limit}$): The access is out of bounds. The MMU generates an illegal memory access trap to the Operating System (Segmentation Fault).
 - If **True**: The address is valid, and translation proceeds.
3. **Address Mapping:** The physical address is computed as:

$$\text{Physical Address} = \text{Base Register} + d$$

C. Memory Protection Mechanism:

This hardware check guarantees that a process can only access memory within its assigned boundary [Base , $\text{Base} + \text{Limit}$), preventing processes from reading or corrupting operating system memory or other active user processes.

Question 3: Concept of Paging and Memory Allocation

A. Core Concept:

Paging is a non-contiguous memory management scheme where physical RAM is partitioned into fixed-size blocks called **frames**, and logical memory is divided into equal-sized blocks called **pages**.

B. Mapping Mechanism:

The OS maintains a process-specific **Page Table**. When a process executes, its pages are loaded into any available physical frames in RAM, regardless of physical contiguity.

C. External vs. Internal Fragmentation:

- **Eliminating External Fragmentation:** Since memory is allocated in uniform, fixed-size chunks, any free frame can accommodate any page. No unallocated gaps are wasted between allocations.
- **Internal Fragmentation:** Can still occur in the last page of a process if the program's memory requirement is not an exact multiple of the page size (averaging half a page per process).

Question 4: Structure and Working of a Page Table

A. Page Table Structure & Entry Fields (PTE):

- **Frame Number (f):** Stores the physical frame index in RAM.
- **Valid / Invalid Bit:** Identifies whether the page is currently in RAM (1) or on secondary disk storage (0).
- **Protection Bits:** Defines Read, Write, and Execute permissions ($R / W / X$).
- **Dirty Bit:** Indicates if the page was modified while residing in main memory.

B. Address Translation Mechanics:

A CPU logical address consists of two parts: Page Number (p) and Offset (d).

1. Use p as an index into the Page Table to retrieve Frame Number f .

2. Combine frame number f with offset d to form the physical address: $\langle f, d \rangle$.

C. Advantages and Limitations:

- **Advantages:** Allows non-contiguous physical allocation, eliminates external fragmentation, and enables virtual memory.
- **Limitations:** Consumes memory space to store page tables; doubles memory access time (one access for table lookup, one for target data).

Question 5: Translation Lookaside Buffer (TLB)

A. Concept:

The TLB is a high-speed associative hardware cache embedded within the MMU that stores recent page-table translations to eliminate memory lookup overhead.

B. Operational Flow:

1. CPU generates page number p .
2. The MMU searches p across all TLB entries simultaneously in parallel.
3. **TLB Hit:** If found, the corresponding frame number f is retrieved instantly without accessing main RAM.
4. **TLB Miss:** If missing, the MMU retrieves f from the Page Table in main memory and updates the TLB for future references.

C. Performance Impact:

By servicing 90%+ of memory address requests directly from hardware cache, the system avoids redundant main memory reads and significantly reduces effective access latency.

Question 6: TLB Hit vs. TLB Miss & Quantitative Performance Analysis

A. Performance Equations:

- **TLB Hit Access Time:** $T_{hit} = \epsilon + m$
- **TLB Miss Access Time:** $T_{miss} = \epsilon + 2m$
- **Effective Access Time (EAT):**

$$EAT = \alpha \cdot (\epsilon + m) + (1 - \alpha) \cdot (\epsilon + 2m)$$

Where α = Hit Ratio, ϵ = TLB Lookup Time, m = RAM Access Time.

B. Numerical Calculation Example:

Assume TLB search time $\epsilon = 10 \text{ ns}$, RAM access time $m = 100 \text{ ns}$, and Hit Ratio $\alpha = 80\% (0.80)$:

- $T_{hit} = 10 + 100 = 110 \text{ ns}$
- $T_{miss} = 10 + 100 + 100 = 210 \text{ ns}$
- $EAT = (0.80 \times 110) + (0.20 \times 210) = 88 + 42 = 130 \text{ ns}$

If hit ratio improves to 98% ($\alpha = 0.98$), $EAT = (0.98 \times 110) + (0.02 \times 210) = 112 \text{ ns}$, yielding a dramatic performance boost.

Question 7: Memory Protection Mechanisms & Access Violations

A. Protection Mechanisms:

- **Hardware Boundary Checking:** Base and Limit registers ensure that processes operate strictly inside their allocated address ranges.
- **Access Control Bits:** Flags in Page Table / Segment Table entries enforce permissions (Read-Only, Read-Write, Execute-Only).

B. Multi-User Environment Importance:

In multi-user systems, protection enforces isolation so that one user's process cannot inspect private data, tamper with running applications, or crash the operating system kernel.

C. Handling Rule Violations:

1. When an unauthorized operation occurs (e.g., writing to a Read-Only code segment or accessing an invalid page), the MMU detects the violation.
2. The hardware raises a high-priority trap to the operating system kernel.
3. The OS handles the trap by terminating the process with an error signal (e.g., Segmentation Fault or General Protection Fault).

Question 8: Concept of Segmentation

A. Logical Concept:

Segmentation is a memory management scheme that views memory from the programmer's perspective. Instead of uniform linear blocks, a program is partitioned into variable-length functional modules called **segments**.

B. Logical Program Structure:

- **Code / Text Segment:** Contains read-only machine instructions.
- **Data Segment:** Stores global and static variables.
- **Stack Segment:** Holds function parameters, local variables, and return addresses.
- **Heap Segment:** Manages dynamically allocated memory at runtime.

C. Logical Addressing:

Addresses are specified as a 2D tuple: $\langle \text{Segment Number } (s), \text{Offset } (d) \rangle$.

D. Benefits: Enables clean modular protection (e.g., marking code as read-only), simplifies code sharing between processes, and eliminates internal fragmentation.

Question 9: Structure and Functioning of a Segment Table

A. Segment Table Structure:

Each entry in the Segment Table maps a segment number s to two primary physical properties:

- **Base:** Starting physical memory location in RAM.
- **Limit:** The exact length of the segment.

B. Address Translation Mechanics:

1. CPU generates logical address tuple $\langle s, d \rangle$.
2. The MMU uses segment index s to look up entry in the Segment Table.
3. **Validity Check:** The MMU verifies if offset $d < Limit$.
 - If $d \geq Limit$: Raises a Segmentation Fault trap.
 - If $d < Limit$: Calculates $Physical\ Address = Base + d$.

Question 10: Comparative Analysis of Paging and Segmentation

The following table summarizes the key structural and operational differences between Paging and Segmentation:

Feature / Parameter	Paging	Segmentation
Viewpoint	Hardware / System view (Fixed physical partitioning)	Programmer view (Logical program modules)
Block Size	Fixed size (e.g., 4 KB)	Variable dynamic length
Address Space	1D linear logical address space	2D address space $\langle Segment, Offset \rangle$
Fragmentation	Internal fragmentation exists; zero external fragmentation	External fragmentation exists; zero internal fragmentation
Mapping Table	Page Table: Page Number $\rightarrow$ Frame Number	Segment Table: Segment Number $\rightarrow$ $\langle Base, Limit \rangle$
Sharing & Protection	Less natural; depends on page alignment	Very natural; applied directly per logical segment